Dynamic Truck–Drone Relief Routing with Full Coverage

Oct 28, 2026 · @Dũng

Abstract

After a flood or an earthquake, relief must reach every affected person, the most urgent first, while roads keep failing and reopening. We study a dynamic two-echelon truck–drone relief routing problem with load-dependent drone energy and full coverage (D2E-TDRP-LD):

- **First echelon.** Trucks carry parcels and drones from a depot to staging satellites over a road network whose roads are open, damaged (×2.5 travel time) or closed (×5, a long detour). Trucks may make several trips, and each satellite is visited at most once.
- **Second echelon.** At each satellite the truck waits while its drones fly multi-drop trips to individual victims. Drone power follows the rotor model of Dorling et al. (2017), which grows with the 3/2 power of frame plus payload mass, so a loaded drone's range is shorter.

Model. We formulate a bi-objective mixed-integer nonlinear program (MO-MINLP) in the style of Yu et al. (2027):

- f_1: priority-weighted average waiting time of victims;
- f_2: fleet activation, truck driving and drone energy cost;
- critical-deadline misses are a constraint violation, so the Pareto front contains only plans that miss as few critical deadlines as possible;
- every victim must be served, and the instance design guarantees this is always feasible.

Because each victim needs one unit, drone loads are whole numbers and the power curve takes only four values, so the model linearizes exactly to a MILP for small instances.

Dynamics. Road states change hourly through a Markov chain. At every hour the plan is re-optimized over what is not yet done or under way: roads trucks have driven or are driving stay frozen, and satellites that a truck has reached keep their drone plans. One plan of the current front is executed according to a decision maker's preference.

Method. A multi-objective adaptive large neighbourhood search (MO-ALNS) with an energy-aware drone split, steered by preference-conditioned double deep Q-networks that choose how much to re-optimize each hour and which operators to apply.

Data. Instances are generated from the Solomon and Gehring & Homberger benchmarks, with 25 to 1,000 victims and 1,110 dynamic scenarios. Every victim lies within single-trip drone range of at least two candidate satellites. Experiments are planned for the next version.

Keywords: humanitarian logistics; two-echelon vehicle routing; truck–drone collaboration; load-dependent energy; multi-objective optimization; dynamic re-planning; deep reinforcement learning.

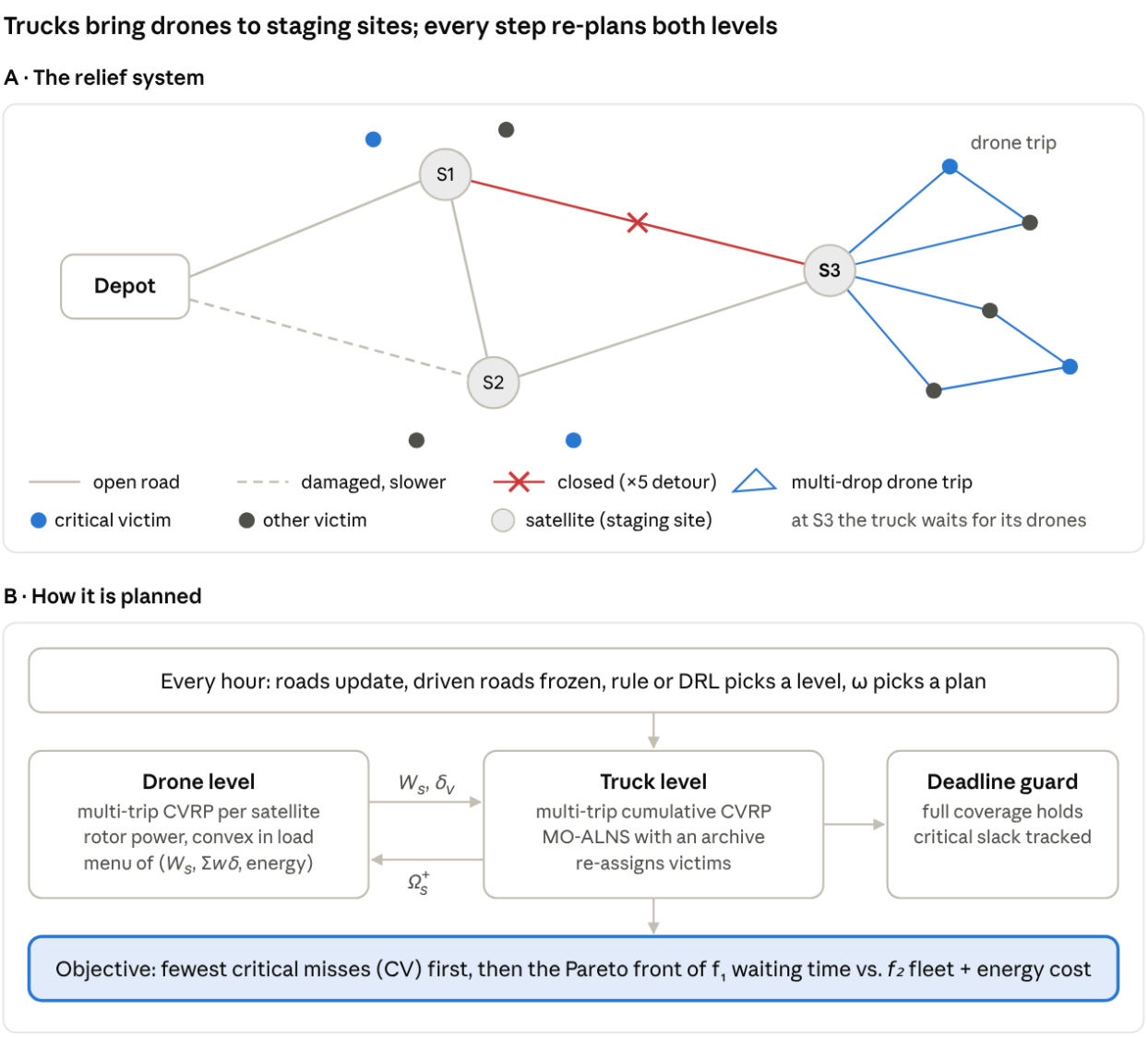


Fig. 1. Overview. Panel A: trucks drive over open, damaged and closed roads to staging satellites, where drones fly multi-drop trips to victims while the truck waits. Panel B: at every time step the drone plans give the truck search its dwell times, and the truck route tells each drone plan how much priority is waiting further along the route.

1. Introduction

In the first hours after a disaster, relief must reach every affected person, and the most urgent first. Damaged roads make both goals hard: a route planned in the morning may cross a bridge that is closed by noon. Drones help because they ignore roads, but they carry only a few kilograms over a few kilometres. In practice, trucks bring supplies and drones to safe staging sites (satellites), and drones fly the last leg.

Two-echelon vehicle–drone routing (2E-VRPD) models this structure. The closest recent model, the 2E-VRPLDTW of Yu et al. (2027), sends drones from satellites to individual customers with a nonlinear, load-dependent power model and optimizes a Pareto front of equipment and operating cost. It is designed for urban last-mile logistics. Three of its assumptions do not hold in relief:

- **Static roads.** The plan is made once, on fixed distances. After a disaster, roads degrade and recover during the operation, and the authors themselves name time-dependent travel conditions as future work.
- **Cost objectives.** Relief is judged by how long people wait, weighted by urgency, not by what the fleet costs.
- **Trucks that do not wait.** Their trucks leave satellites without their drones. When communications are down and drones are scarce, the truck waits for its drones (Nguyen et al.), and that waiting delays every later satellite.

The relief-specific two-echelon model of Nguyen et al. (2E-CTVRP) does make the truck wait, but its drones fly to cluster centroids rather than to victims, energy does not depend on load, and the plan is static.

Gap. No model re-plans a two-echelon truck–drone relief system during execution while (i) drones serve individual victims under a load-dependent energy model, (ii) every victim is guaranteed to be served, and (iii) the decision maker sees the trade-off between waiting time and the resources committed.

Contributions. This paper's main contributions are:

1. **A dynamic, bi-objective two-echelon relief model.** We formulate the D2E-TDRP-LD as an MO-MINLP following the structure of Yu et al. (2027), extended with:
    - multi-trip trucks that wait at satellites for their drones;
    - multi-drop drone trips with the rotor power model (Dorling et al. 2017);
    - a waiting-time objective against a resource-cost objective, with critical deadlines as constraint violation.

Unit demands make the power curve piecewise-constant, so the model has an exact MILP form for small instances. 2. **Re-planning on a degrading road network.** Roads change hourly through a Markov chain with three states. A rolling-horizon procedure freezes driven roads and reached satellites, re-optimizes truck routes, victim assignments and drone plans together, and executes one plan of the current front by preference. Full coverage is feasible at every re-plan by construction. 3. **A learning-assisted multi-objective ALNS.** An energy-aware split solves each satellite's drone plan inside an archive-based MO-ALNS. Preference-conditioned double DQNs choose how much each hour's re-plan deserves and which operators to apply; they never build routes, so every plan stays feasible. 4. **A dynamic relief benchmark.** 37 instances from Solomon and Gehring & Homberger (25 to 1,000 victims), with priority classes, double-coverage satellite candidates, a road graph, a flood zone and 30 road-damage histories each.

Organization. Section 2 reviews related work. Section 3 defines the problem, presents the MO-MINLP and its dynamic extension. Section 4 describes the solution method. Section 5 describes the benchmark and the experimental design. Section 6 concludes.

2. Related work

Our model joins five lines of work. Table 1 summarizes the studies closest to it.

2.1 Two-echelon vehicle–drone routing

- **Nguyen et al. (2E-CTVRP)** send trucks to satellites, where each drone makes one round trip to the centroid of a victim cluster and the truck waits for its drones. The objective is the sum of truck arrival times, close to the cumulative CVRP (Ngueveu et al. 2010). The plan is static.
- **Yu et al. (2027, 2E-VRPLDTW)** send drones from satellites to individual pickup or delivery customers with time windows. Drone power follows the rotor model of Dorling et al. (2017), drones may return to the nearest satellite, and trucks do not wait. A constrained multi-objective differential evolution (DSDE) approximates the front of equipment and operating cost.
- **Shi et al. (2026)** feed satellites with large trucks and serve sites with small trucks that each carry a drone, against budgeted uncertainty in demand and travel time. Their bi-objective model (completion time, cost) is solved once, in advance.

Other truck–drone papers launch drones from customer nodes on the truck route (Meng & Ma 2026; Luo et al. 2027), which suits parcel delivery more than relief.

2.2 Coverage of every demand point in relief

Approaches differ in how strongly they guarantee service:

Approach	Papers	Limitation for a dynamic disaster
Hard constraint at planning time	Nguyen et al.; most VRP-D models	Holds for the t = 0 plan only; no repair after roads fail
Penalty for unserved demand (equity)	Peng et al. 2025	Soft: a cheap enough plan may still leave people out
Outsourcing at high cost	Luo et al. 2027	Always feasible, but only as a cost
Drone-only sites for unreachable points	Peng et al. 2025; Shi et al. 2026; Du et al. 2026	Which sites are drone-only is fixed in advance
Hard time windows	Yu et al. 2027	Large preparation times can make instances infeasible

We keep the hard rule and make it hold by construction, at planning time and during execution (Section 3.2.5).

2.3 Load-dependent drone energy and multi-trip drones

Three energy models are common:

- **Distance or time only:** fixed endurance (Peng et al. 2025; Shi et al. 2026).
- **Linear in mass × time:** Meng & Ma (2026) and Luo et al. (2027), with multi-drop trips whose load falls after each drop.
- **Rotor power:** $P = (W + y)^{3/2} \sqrt{g^3 / 2\rho\zeta n}$ (Dorling et al. 2017), used by Yu et al. (2027). It is convex in the load, so heavy multi-drop trips are penalized more than under a linear model.

We adopt the rotor model and Meng & Ma's instant battery swap with spare batteries.

2.4 Multi-objective vehicle–drone routing

Recent studies optimize Pareto fronts rather than weighted sums:

- Yu et al. (2027): equipment cost vs. operating cost, by DSDE with a dynamic constraint tolerance;
- Shi et al. (2026): completion time vs. cost, by NSGA-II with tabu search, validated with the ε-constraint method;
- Duan et al. (2026): cost vs. time-window violation, by a multi-objective hyper-heuristic memetic algorithm.

All three solve a static problem once. In a dynamic problem the front must be maintained over time and one plan must be executed at each step; we follow the decomposition idea of MOEA/D (Zhang & Li 2007) and the constrained dominance of NSGA-II (Deb et al. 2002) inside an ALNS (Ropke & Pisinger 2006).

2.5 Dynamic re-planning and learning

- **Dynamic truck–drone relief.** Peng et al. (2025) re-plan every 15 minutes with multi-agent deep RL in a single echelon, with new demands rather than road changes.
- **Learned routing policies.** Chen, Ulmer & Thomas (2022) learn with deep Q-learning whether a truck or a drone serves each request. Li & Gonsalves (2026) learn to construct TSP-D and VRP-D routes.
- **Learned control of a search.** Ropke & Pisinger (2006) adapt operator weights, and Duan et al. (2026) select heuristics. For several objectives, a preference-conditioned Q-network serves all preferences with one policy (Abels et al. 2019).

We follow the last line: learning decides how to re-optimize, and the optimizer builds the routes and guarantees feasibility.

2.6 Positioning

Table 1. Studies closest to ours.

Study	Satellites (two echelons)	Victim-level multi-drop drones	Load-dependent energy	Pareto front	Re-planning during execution	Learning
Nguyen et al. (2E-CTVRP)	Yes	No (one centroid per trip)	No	No	No	No
Yu et al. 2027	Yes	Yes	Rotor power	Yes (equipment vs. operating cost)	No	No
Shi et al. 2026	Yes	No (one site per flight)	No	Yes (time vs. cost)	No (robust)	No
Meng & Ma 2026	No	Yes	Linear	No	No	No

Study	Satellites (two echelons)	Victim-level multi-drop drones	Load-dependent energy	Pareto front	Re-planning during execution	Learning
Luo et al. 2027	No	Yes	Linear	No	No	No
Duan et al. 2026	No	Yes	Load-dependent	Yes (cost vs. violation)	No	Hyper-heuristic
Peng et al. 2025	No	No (one area per flight)	No	No	Yes (15 min)	Multi-agent DRL
This paper	**Yes**	**Yes**	**Rotor power**	**Yes (waiting vs. resource cost)**	**Yes (hourly roads)**	**Double DQN on top of MO-ALNS**

To our knowledge, no model combines satellites with a waiting truck and multi-trip trucks, victim-level drones under rotor power, a waiting-versus-resource front, hourly Markov road damage with frozen roads, guaranteed full coverage, and learned re-planning control.

3. Problem definition and formulation

Section 3.1 defines the problem and its notation. Section 3.2 gives the decision variables and the MO-MINLP at one decision time, with road states known. Section 3.3 extends it to re-planning while roads change.

3.1 Problem definition and notation

A relief operation starts at time 0 from one depot. Trucks carry parcels and drones from the depot to staging satellites; at each satellite the truck stops and its drones fly multi-drop trips that deliver one parcel to every victim assigned there; the truck leaves when its last drone has landed. Victims belong to priority classes, all of them must be served, and critical victims have deadlines. The goal is a plan that serves victims early, weighted by urgency, while committing few vehicles and little energy.

Graph. The problem is defined on the node set $N = \{0\} \cup S \cup V$, with depot 0, candidate satellites S and victims V.

- **Truck arcs** join nodes of $\{0\} \cup S$. Trucks drive on a road graph over these nodes and road junctions, so $c_{ij}(t)$ is the fastest truck time from i to j at the road states of time t.
- **Drone arcs** join nodes of $S \cup V$. Drones fly straight lines of length d_{ij} km, taking $\hat{t}_{ij} = 60\, d_{ij} / v_a$ minutes.

Road states. Each road has a base time at $v_t = 40$ km/h and a state:

State	Travel time	Meaning
Open	×1	Normal
Damaged	×2.5	Usable but slow (debris, shallow water, one lane)
Closed	×5	Blocked; the truck takes a long local detour

No road is impassable, so every $c_{ij}(t)$ is finite and every satellite is always reachable.

Priority classes.

Class	Weight w_v	Target D_v	Treatment
Critical (injured, medical needs)	5	120 min	Deadline; a miss is a constraint violation
High (children, elderly, isolated)	2	240 min	Soft target, lateness penalized
Normal	1	480 min	Soft target, lateness penalized

Drone power. Following Dorling et al. (2017) and Yu et al. (2027), a drone carrying y units of q kg draws the power

$$P(y) = (W + q\,y)^{3/2}\sqrt{\frac{g^3}{2\,\rho\,\zeta\,n_r}}$$

where W is the drone's own mass with battery (kg), g the gravitational acceleration, ρ the air density, ζ the disc area of one rotor (m²) and n_r the number of rotors. The power is the same in cruise and in hover, and it depends only on the load on board. As a fraction of the battery E per minute:

$$p(y) = \frac{P(y)}{60\,E}$$

We calibrate a heavy-lift hexacopter: W = 6 kg, n_r = 6, ζ = 0.26 m² (rotors of about 58 cm), E = 180 Wh, q = 1 kg. At 60 km/h one minute equals one kilometre, so:

Units on board	Power	Drain per km	Distance on 80% of the battery	Previous linear model
0	231 W	2.14%	37 km	2.22%
1	291 W	2.70%	30 km	2.59%
2	356 W	3.29%	24 km	2.96%
3	424 W	3.93%	20 km	3.33%

The calibration keeps the single-unit range of our earlier linear model, but the drain grows faster with load: a full drone drains 84% more than an empty one, against 50% under the linear model.

Coverage. Satellite s covers victim v (a_{vs} = 1) if a trip that carries one unit to v and returns empty is battery-feasible, that is, if $d_{sv} \le R_1$ with

$$R_1 = \frac{v_a}{60}\cdot\frac{1 - r_{\min} - p(1)\,\kappa}{p(1) + p(0)} \approx 16.5\ \text{km}$$

Coverage depends only on fixed locations and the drone, so a_{vs} is computed once per instance. Road damage affects only the trucks.

Assumptions. The following assumptions describe the operation:

1. Trucks leave the depot with parcels and drones, visit satellites, and may return to reload and leave again (multi-trip, reload time θ = 0). Each satellite is visited at most once over the whole operation.

2. At a satellite the truck waits until all its drones have landed. Drones launch and land at their own truck's satellite: with communications down and drones scarce, a truck does not leave drones behind (Nguyen et al.). This replaces the cross-satellite recovery of Yu et al. (2027).
3. Every victim needs one unit of q = 1 kg and is served once, by one drop. A truck carries at most $Q_1 = 500$ units per trip and a drone at most $Q_2 = 3$ units per trip. Drones and batteries do not count toward Q_1.
4. Trucks and drones drive and fly at constant speeds (40 and 60 km/h). Drones are unaffected by road states.
5. Each drone trip starts on a full battery: batteries are swapped instantly from an unlimited stock. Launch and recovery take no time; each drop takes a hover of $\kappa = 0.1$ min at the load before the drop.
6. A trip must land with at least $r_{min} = 20\%$ of its battery.
7. A victim may be served only from a satellite that covers it.
8. Missing a critical deadline is a constraint violation, minimized before the objectives. Lateness of the other classes is penalized in f_1.
9. Up to $|K|$ trucks, each able to carry $\bar{U}$ drones, are available at the depot. Activating a truck or a drone has a fixed cost, and an idle truck may be activated at a later re-plan.
10. Truck operating cost is proportional to driving time, and drone operating cost to energy.
11. Road states change only at hourly boundaries and are known when they change (Section 3.3).

Table 2. Notation.

Symbol	Meaning	Value
$N = \{0\} \cup S \cup V$	Depot, candidate satellites, victims	Instance
$V^c \subseteq V$	Critical victims	20% of V
K	Trucks available at the depot	Workload rule (2 to 7)
U, U_k	All drones; drones carried by truck k	$\bar{U} = 3$ per truck
$\bar{R}$, $\bar{H}$	Maximum trips per truck, per drone	Upper bounds
$c_{ij}(t)$	Fastest truck time from i to j at time t (min)	Road graph
d_{ij}, $\hat{t}_{ij}$	Drone distance (km) and flight time (min)	Euclidean
a_{vs}	1 if satellite s covers victim v	$d_{sv} \le R_1$
w_v, D_v	Priority weight and target time of victim v	Priority table
Q_1, Q_2, q	Truck capacity, drone payload (units), unit mass	500, 3, 1 kg
v_t, v_a	Truck and drone speeds	40, 60 km/h
κ, θ	Drop hover time, truck reload time	0.1 min, 0
W, n_r, ζ	Drone mass with battery, rotors, rotor disc area	6 kg, 6, 0.26 m^2
g, ρ	Gravity, air density	9.81 m/s^2, 1.225 kg/m^3
E, r_{min}	Battery capacity, landing reserve	180 Wh, 20%
$h^{(1)}$, $h^{(2)}$	Activation cost of a truck, of a drone	12.5 $, 3.36 $
c_t, c_a	Truck driving cost, drone energy cost	0.0086 $/min, 0.0001 $/Wh

Symbol	Meaning	Value
π_l	Weight of a minute of lateness	2
M	Large constant (time)	Horizon

The cost parameters are those of Yu et al. (2027), adopted from Jung (2025); their truck cost of 0.01286 $/km becomes 0.0086 $ per driving minute at 40 km/h, so damaged roads cost more to drive.

3.2 Decision variables and formulation

The decision variables are defined in Table 3. Trucks are indexed by k and their trips by r; drones by u and their trips by h. A truck trip leaves the depot, visits satellites and returns; a drone trip leaves a satellite, visits victims and lands at the same satellite.

Table 3. Decision variables.

Variable	Definition
$\tau_k \in \{0, 1\}$	Truck k is activated
$\mu_u \in \{0, 1\}$	Drone u is activated
$\psi^{kr} \in \{0, 1\}$	Truck k makes trip r
$m_s^{kr} \in \{0, 1\}$	Truck k visits satellite s on trip r
$x_{ij}^{kr} \in \{0, 1\}$	Truck k drives from i to j on trip r ($i, j \in \{0\} \cup S$)
$g_s^{kr} \geq 0$	Units truck k unloads at satellite s on trip r
$\sigma_{vs} \in \{0, 1\}$	Victim v is assigned to satellite s
$o_s^{uh} \in \{0, 1\}$	Trip h of drone u launches from satellite s
$\xi_{ij}^{uh} \in \{0, 1\}$	Trip h of drone u flies from i to j ($i, j \in S \cup V$, not both satellites)
$n_v^{uh} \in \{0, 1\}$	Victim v is served on trip h of drone u
$y_{ij}^{uh} \in \{0, \ldots, Q_2\}$	Units on board on arc (i, j) of that trip
$e_{ij}^{uh} \geq 0$	Energy (Wh) on arc (i, j), including the drop hover at j
$b_i^{uh} \in [0, 1]$	Battery fraction left after node i on that trip
A_s, L_s	Truck arrival at satellite s, and departure after its dwell
α^{kr}, ω^{kr}	Start and return times of trip r of truck k
β^{uh}, γ^{uh}	Launch and landing times of trip h of drone u
T_v	Time victim v receives its parcel
$\ell_v \geq 0$	Lateness of victim v against its target D_v
$\chi_v \in \{0, 1\}$	Critical victim v misses its deadline

The dwell at satellite s is $W_s = L_s - A_s$. All times are in minutes.

3.2.1 Objective functions

Minimize

$$f_1 = \frac{1}{\sum_{v \in V} w_v} \sum_{v \in V} w_v \left(T_v + \pi_l \, \ell_v \right) \tag{1}$$

$$f_2 = \sum_{k \in K} h^{(1)} \tau_k + \sum_{u \in U} h^{(2)} \mu_u + c_t \sum_{k,r} \sum_{i,j} c_{ij}\, x_{ij}^{kr} + c_a \sum_{u,h} \sum_{i,j} e_{ij}^{uh} \tag{2}$$

subject to a minimum constraint violation on critical deadlines:

$$CV = \sum_{v \in V^c} \left(\chi_v + \frac{\ell_v}{D_v} \right) \tag{3}$$

- **f₁, service.** The priority-weighted average time victims wait for their parcel, with each minute past a soft target counted 1 + π₁ = 3 times.
- **f₂, resources.** The cost of the trucks and drones activated (Yu et al.'s equipment objective), plus truck driving and drone energy (their operating objective). We merge their two cost objectives into one because, in relief, the competing goal is waiting time.
- **CV, critical deadlines.** The number of critical victims served late plus their relative lateness.

The problem is to find the Pareto front of (f₁, f₂) over the plans with the smallest CV:

$$\min_{x \in X} \; F(x) = \big(f_1(x),\, f_2(x) \big) \quad \text{s.t.} \quad CV(x) = CV^* = \min_{x' \in X} CV(x')$$

where X is the set of plans satisfying constraints (4)–(38). Plan x dominates plan x′ if CV(x) < CV(x′), or if CV(x) = CV(x′) and x is no worse in both objectives and better in one. This is the constrained dominance of Deb et al. (2002). Since CV* = 0 whenever every critical deadline can be met, the deadlines then act as hard constraints; when road damage makes that impossible, the front contains the plans that miss as few deadlines, by as little, as possible.

Why two objectives. Following the reasoning of Yu et al. (2027):

1. Minutes of waiting and dollars of fleet cost have no agreed exchange rate in relief, so a fixed weight is hard to justify across disasters.
2. The trade-off is real. Every extra truck or drone committed here shortens dwells and routes, but it is taken from another affected area.
3. The front tells the decision maker, for example, how many weighted minutes of waiting one more truck saves.

3.2.2 Routing constraints

$$\sum_{k \in K} \sum_{r} m_s^{kr} \le 1 \quad \forall s \in S \tag{4}$$

$$\begin{aligned} \sum_{j \in S} x_{0j}^{kr} &= \psi^{kr} \\ \sum_{i \in S} x_{i0}^{kr} &= \psi^{kr} \quad \forall k, r \end{aligned} \tag{5}$$

$$\sum_{i \in \{0\} \cup S} x_{is}^{kr} = \sum_{j \in \{0\} \cup S} x_{sj}^{kr} = m_s^{kr} \quad \forall s \in S,\ \forall k, r \tag{6}$$

$$\psi^{k,r+1} \le \psi^{kr} \le \tau_k \quad \forall k, r \tag{7}$$

$$\sum_{s \in S} \sigma_{vs} = 1 \quad \forall v \in V \tag{8}$$

$$\begin{aligned} \sigma_{vs} &\le a_{vs} \\ \sigma_{vs} &\le \sum_{k,r} m_s^{kr} \quad \forall v \in V,\ \forall s \in S \end{aligned} \tag{9}$$

$$\sum_{u \in U} \sum_{h} n_v^{uh} = 1 \quad \forall v \in V \tag{10}$$

$$\sum_{i \in S \cup V} \xi_{iv}^{uh} = \sum_{j \in S \cup V} \xi_{vj}^{uh} = n_v^{uh} \quad \forall v \in V,\ \forall u, h \tag{11}$$

$$\sum_{v \in V} \xi_{sv}^{uh} = \sum_{v \in V} \xi_{vs}^{uh} = o_s^{uh} \quad \forall s \in S,\ \forall u, h \tag{12}$$

$$\begin{aligned} \sum_{s \in S} o_s^{uh} &\le \mu_u \\ \sum_{s \in S} o_s^{u,h+1} &\le \sum_{s \in S} o_s^{uh} \quad \forall u, h \end{aligned} \tag{13}$$

$$n_v^{uh} \le \sigma_{vs} + 1 - o_s^{uh} \quad \forall v \in V,\ \forall s \in S,\ \forall u, h \tag{14}$$

$$\begin{aligned} o_s^{uh} &\le \sum_{r} m_s^{kr} \\ \mu_u &\le \tau_k \quad \forall k,\ \forall u \in U_k,\ \forall s, h \end{aligned} \tag{15}$$

The routing constraints coordinate trucks, satellites and drones.

- **Trucks.** Constraint (4) lets each satellite be visited at most once over all trucks and trips, so there are no revisits. Constraints (5) make each trip leave and return to the depot, and (6) link satellite visits to the arcs of a trip. Constraint (7) uses a truck's trips in order and only if the truck is activated.
- **Full coverage.** Constraint (8) assigns every victim to exactly one satellite, and (9) allows only covering satellites that some truck visits. Together with (10), every victim is served.
- **Drones.** Constraint (10) serves each victim once, on one drone trip, and (11) gives it one incoming and one outgoing arc on that trip. Constraint (12) makes a trip launch and land at the same satellite, unlike the cross-satellite recovery of Yu et al. (2027). Constraint (13) uses a drone's trips in order and only if the drone is activated.
- **Echelon link.** Constraint (14) lets a trip launched at s serve only victims assigned to s. Constraint (15) launches a drone only at satellites its own truck visits, and activates a drone only on an activated truck.

Subtours are removed by the time constraints of Section 3.2.3, since every truck and drone arc takes positive time.

3.2.3 Time constraints

$$A_j \geq \alpha^{kr} + c_{0j} - M\,(1 - x_{0j}^{kr}) \quad \forall j \in S,\ \forall k, r \tag{16}$$

$$A_j \geq L_i + c_{ij} - M\,(1 - x_{ij}^{kr}) \quad \forall i, j \in S,\ \forall k, r \tag{17}$$

$$\omega^{kr} \geq L_i + c_{i0} - M\,(1 - x_{i0}^{kr}) \quad \forall i \in S,\ \forall k, r \tag{18}$$

$$\alpha^{k,r+1} \geq \omega^{kr} + \theta, \qquad \alpha^{k1} \geq 0 \quad \forall k, r \tag{19}$$

$$L_s \geq A_s \quad \forall s \in S \tag{20}$$

$$\beta^{uh} \geq A_s - M\,(1 - o_s^{uh}) \quad \forall s \in S,\ \forall u, h \tag{21}$$

$$\beta^{u,h+1} \geq \gamma^{uh} \quad \forall u, h \tag{22}$$

$$T_v \geq \beta^{uh} + \hat{t}_{sv} - M\,(1 - \xi_{sv}^{uh}) \quad \forall s \in S,\ \forall v \in V,\ \forall u, h \tag{23}$$

$$T_j \geq T_i + \kappa + \hat{t}_{ij} - M\,(1 - \xi_{ij}^{uh}) \quad \forall i, j \in V,\ \forall u, h \tag{24}$$

$$\gamma^{uh} \geq T_i + \kappa + \hat{t}_{is} - M\,(1 - \xi_{is}^{uh}) \quad \forall i \in V,\ \forall s \in S,\ \forall u, h \tag{25}$$

$$L_s \geq \gamma^{uh} - M\,(1 - o_s^{uh}) \quad \forall s \in S,\ \forall u, h \tag{26}$$

$$\ell_v \geq T_v - D_v, \qquad \ell_v \geq 0 \quad \forall v \in V \tag{27}$$

$$T_v \leq D_v + M\,\chi_v \quad \forall v \in V^c \tag{28}$$

The time constraints synchronize the two echelons.

- **Truck times.** Constraints (16)–(18) propagate arrivals along each trip, using the dwell end L_i of the previous satellite. Constraint (19) starts each trip after the previous one returns (reload $\theta = 0$).
- **Launch after arrival.** Constraint (21) launches a drone trip at s only after its truck has arrived there, and (22) starts a drone's next trip as soon as it lands, because batteries are swapped instantly.
- **Drone times.** Constraints (23)–(25) propagate drop and landing times along each drone trip, with a hover of κ at each drop.
- **Truck waits for its drones.** Constraint (26) is the key coupling of our model. The truck leaves s only after the last drone launched there has landed, so the dwell W_s delays every later satellite of the truck. Yu et al. (2027) synchronize only the launch, as in (21); their trucks do not wait.
- **Targets and deadlines.** Constraint (27) measures lateness against the soft targets, and (28) flags the critical victims whose deadline is missed, which enter CV.

3.2.4 Load and energy constraints

$$\begin{gathered} g_s^{kr} \geq \sum_{v \in V} \sigma_{vs} - Q_1 (1 - m_s^{kr}) \\ g_s^{kr} \geq 0 \\ \sum_{s \in S} g_s^{kr} \leq Q_1 \, \psi^{kr} \quad \forall k, r \end{gathered} \tag{29}$$

$$y_{ij}^{uh} \leq Q_2 \, \xi_{ij}^{uh} \quad \forall i, j \in S \cup V, \; \forall u, h \tag{30}$$

$$\sum_{i \in S \cup V} y_{iv}^{uh} - \sum_{j \in S \cup V} y_{vj}^{uh} = n_v^{uh} \quad \forall v \in V, \; \forall u, h \tag{31}$$

$$y_{vs}^{uh} = 0 \quad \forall v \in V, \; \forall s \in S, \; \forall u, h \tag{32}$$

$$P_{ij}^{uh} = \left(W + q \, y_{ij}^{uh} \right)^{3/2} \sqrt{\frac{g^3}{2 \rho \zeta n_r}} \quad \forall i, j \in S \cup V, \; \forall u, h \tag{33}$$

$$e_{ij}^{uh} = \frac{P_{ij}^{uh}}{60} \left(\hat{t}_{ij} + \kappa \, 1[\, j \in V \,] \right) \xi_{ij}^{uh} \quad \forall i, j \in S \cup V, \; \forall u, h \tag{34}$$

$$b_s^{uh} = 1 \quad \forall s \in S, \; \forall u, h \tag{35}$$

$$b_j^{uh} \leq b_i^{uh} - \frac{e_{ij}^{uh}}{E} + \left(1 - \xi_{ij}^{uh} \right) \quad \forall i \in S \cup V, \; \forall j \in V, \; \forall u, h \tag{36}$$

$$b_i^{uh} - \frac{e_{is}^{uh}}{E} \geq r_{\min} - \left(1 - \xi_{is}^{uh} \right) \quad \forall i \in V, \; \forall s \in S, \; \forall u, h \tag{37}$$

$$0 \leq b_i^{uh} \leq 1 \quad \forall i \in S \cup V, \; \forall u, h \tag{38}$$

The load and energy constraints track parcels and battery.

- **Truck load.** Constraint (29) loads each truck trip with all the units of the satellites it visits and caps the trip at Q_1.
- **Drone load.** Constraint (30) caps a drone's load at $Q_2 = 3$ units. Constraint (31) drops one unit at each victim served, and (32) makes every trip land empty, so the load falls by one after each drop.
- **Power and energy.** Constraint (33) is the rotor power at the current load, the nonlinear part of the model, as in Yu et al. (2027). Constraint (34) is the energy of an arc: flight time plus the drop hover at j, both at the load carried into j.
- **Battery.** Constraint (35) starts every trip on a full battery (instant swap). Constraint (36) subtracts each arc's energy, with the big-M equal to 1 because b is a fraction, and (37) keeps $r_{\min} = 20\%$ at landing.

Exact linear form. Unlike Yu et al. (2027), whose payloads are continuous, our loads take only the values 0, 1, 2 and 3. With binaries λ_{ijl}^{uh} for "load l on arc (i, j)", constraint (33) becomes linear:

$$\sum_{l=0}^{Q_2} \lambda_{ijl}^{uh} = \xi_{ij}^{uh}$$

$$y_{ij}^{uh} = \sum_{l=0}^{Q_2} l\, \lambda_{ijl}^{uh}$$

$$e_{ij}^{uh} = \sum_{l=0}^{Q_2} \frac{P(l)}{60} \left(\hat{t}_{ij} + \kappa\, 1[\, j \in V\,] \right) \lambda_{ijl}^{uh}$$

The model is then a MILP that Gurobi can solve exactly on small instances (Section 5.4.1).

How a trip is checked. Inside the heuristic, constraints (30)–(38) reduce to one test per candidate trip. A trip is a satellite s and an ordered list of victims (v_1, …, v_m), m ≤ 3, flown on a full battery. Let ℓ_0 be the leg from s to v_1, ℓ_i the leg from v_i to v_{i+1}, and ℓ_m the leg back to s, in km (one km is one minute at 60 km/h).

1. **Units on board.** The drone leaves with m units, carries m − i after the i-th drop, and flies home empty.
2. **Energy of each part.** Each leg uses p(units on board) × its length; each drop uses p(units before the drop) × κ.
3. **Total.**

$$e_\rho = \sum_{i=0}^{m} p(m-i)\, \frac{60\,\ell_i}{v_a} + \sum_{i=1}^{m} p(m-i+1)\, \kappa$$

4. **Decision.** The trip is feasible if $e_\rho \le 1 - r_{\min} = 0.80$.

Worked examples. A trip that flies 6 km out, drops at three victims 3 km apart and returns 7 km takes 19.3 min and uses 57.5% of the battery (103.5 Wh); the linear model gave 53%. A harder case: the first victim is 14 km out, the other two lie 1 km apart beyond it, and the way back is 14.5 km.

Trip option	Time	Rotor power model	Previous linear model
All three victims in one trip	30.8 min	93.0%, infeasible	85.4%, infeasible
Two victims (14 km out, 2 km between, 14.5 km back)	30.7 min	83.1%, infeasible	79.5%, feasible
One victim (14 km out and back)	28.1 min	67.9%, feasible	67.7%, feasible

Under the rotor model the far victims get single-unit trips, flown in parallel by the three drones. The linear model would have accepted a two-victim trip that, under the rotor model, lands with only 17% of the battery. Reference ranges at 60 km/h: a single-unit trip reaches $R_1 \approx 16.5$ km, a two-unit trip to a cluster about 14.6 km away, and a full three-unit trip a cluster about 13.0 km away.

Tracking the battery during execution. Each accepted trip stores its battery level after every leg and drop. The simulator starts every trip at 100% and subtracts the same

energies as the drone flies; since drones do not use roads and fly at constant speed, a checked trip cannot run out mid-flight. The simulator logs the lowest level of each trip, so the experiments can report how close trips come to the reserve.

3.2.5 Model properties

Cumulative form. Write truck k's whole sequence, across all its trips, as stops $\pi_1, \ldots, \pi_m$, where a depot reload is a stop of dwell θ and weight 0. With $\Omega_s = \Sigma_v\, w_v\, \sigma_{vs}$ the priority weight at s, $\Omega_{\geq p}$ the weight from stop p onward, and $\delta_v = T_v - A_s$ the drone offset of victim v at its satellite:

$$\sum_v w_v T_v = \sum_p \Omega_{\geq p}\, c_{\pi_{p-1}\pi_p} + \sum_p \Omega_{>p}\, W_{\pi_p} + \sum_v w_v\, \delta_v$$

This has three consequences:

1. **Known problem class.** With drone plans fixed, the truck problem is a weighted, multi-trip cumulative CVRP with service times (Ngueveu et al. 2010), so its moves and bounds apply.
2. **Ordering rule.** Ignoring travel, two neighbouring satellites i, j should swap when $\Omega_j / W_j > \Omega_i / W_i$ (Smith 1956). For the same reason a reload delays everything after it, so heavy-priority satellites belong on a truck's first trip.
3. **Dwell matters through the route.** A satellite's dwell W_s does not delay its own truck arrival, but it delays every later stop, at a cost of $\Omega_{>p} \times W_s$. Drone plans must therefore be judged at their route position, which the drone solver does (Section 4.1).

Proposition 1 (full coverage is always feasible). Suppose that every victim is covered by some satellite, that some covering assignment gives each used satellite at most Q_1 victims, and that all road times are finite. Then, for any activated fleet with at least one truck and one drone, a plan satisfying (4)–(38) exists.

Proof. Serve each used satellite on its own truck trip; trips run back to back, so one truck suffices. Each trip is within capacity. Each victim gets its own single-unit drone trip, which is battery-feasible because the satellite covers it. Every trip ends in finite time. □

The generator verifies the first two conditions (Section 5.1.1), and closed roads at ×5 give the third. Full coverage therefore never needs a fallback; only timeliness and cost are at stake. It also bounds the front: the cheapest plan uses one truck and one drone, but constrained dominance keeps it only if it meets as many critical deadlines as the full fleet can.

Proposition 2 (preference-optimal plans lie on the front). For any weights $\omega > 0$, a plan minimizing the weighted Tchebycheff function of Section 3.3.4 over the plans with CV = CV* is weakly Pareto-optimal, and every Pareto-optimal plan minimizes it for some ω (Miettinen 1999). Executing the archive member chosen by ω is therefore consistent with the front.

3.3 Dynamic extension: re-planning on a degrading road network

The model of Section 3.2 describes one decision time with road states known. During the operation, roads change, so the plan is re-optimized at every step boundary $t_e = e \cdot \Delta t$ (Δt = 1 h, 30 min as a variant), the only times road states change. Over an operation of 8–12 h this

means 8–12 re-plans. Each re-plan keeps what is done or under way, re-solves the rest as a residual instance of (1)–(38), and executes one plan of the new front (Fig. 2).

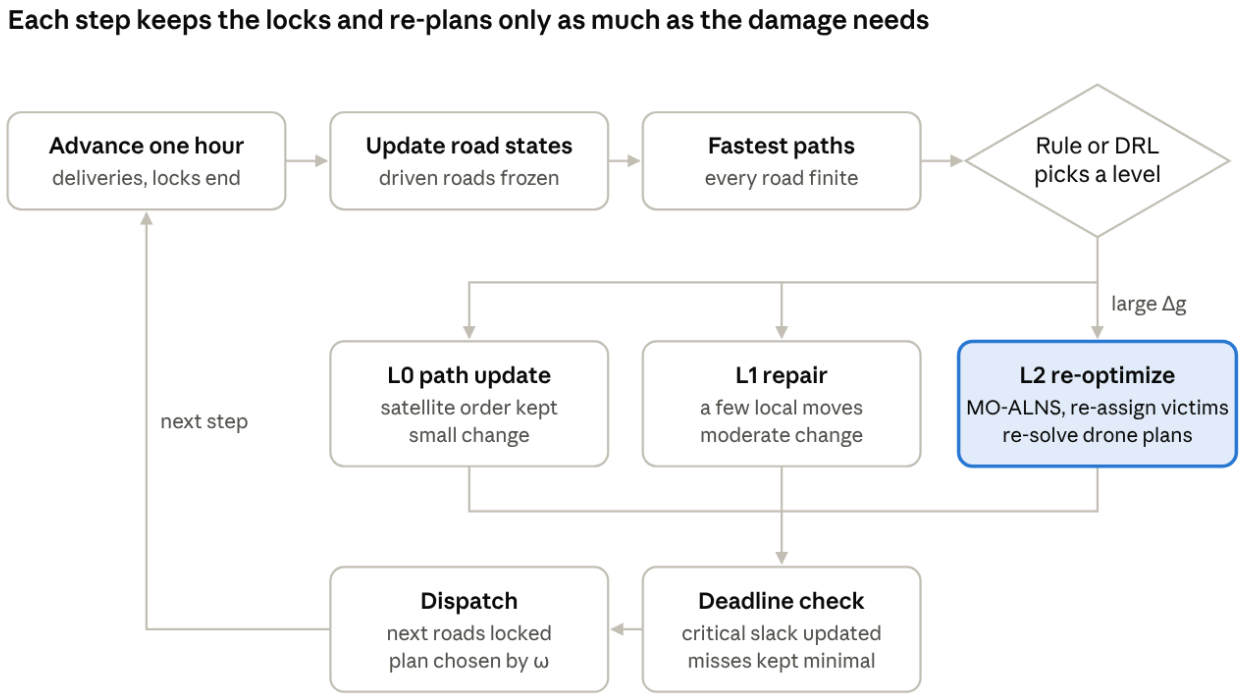


Fig. 2. One time step. Road changes are applied, the current plan is evaluated under the new times, and one of three re-planning levels is chosen; only a large loss triggers the full multi-objective re-optimization. Section 4.4 replaces the fixed rule with a learned policy.

3.3.1 Road dynamics

Flood zone.

- **Centre:** the location of a randomly chosen victim, jittered by up to 5% of the side and clipped to the map, so it always lies inside the area and where people are.
- **Radius:** set so that about 30% of the roads are in the zone. A road is in the zone if any part of it lies within the radius, so a road with only one end inside, or one that merely passes through, is affected too.
- Roads outside the zone stay open.

Initial state. At t = 0 each zone road's state is drawn from the long-run mix of the chosen intensity (last column below), so the first plan already faces damage.

Hourly transitions. At each boundary, every zone road that is not frozen moves by a Markov matrix. Rows are "from" open, damaged, closed; columns are "to" the same states:

Intensity	Hourly matrix	Long-run open / damaged / closed
Low	[[0.95, 0.04, 0.01], [0.30, 0.66, 0.04], [0.13, 0.13, 0.74]]	83% / 12% / 6%
Medium	[[0.88, 0.08, 0.04], [0.28, 0.63, 0.09], [0.17, 0.13, 0.70]]	65% / 20% / 15%

Intensity	Hourly matrix	Long-run open / damaged / closed
High	[[0.80, 0.11, 0.09], [0.23, 0.59, 0.18], [0.12, 0.13, 0.75]]	45% / 23% / 32%

A closure lasts about 3 hours on average, and closed roads can reopen. For the 30-minute variant the matrices are replaced by ones with the same long-run shares.

Frozen roads. A road that any truck has driven, or is driving at a boundary, keeps its state from then on. Changes never affect a truck mid-road, and a truck returning along a road it used keeps the time it saw. After each update the fastest-path times $c_{ij}(t_e)$ are recomputed with Dijkstra; all are finite.

3.3.2 State and locks at a boundary

The state at t_e is the road states, the victims served so far, the assignment of unserved victims, each drone's ongoing trip, the activated fleet, and each truck's mode, remaining parcels and free point (where and when its plan may change):

Truck mode at t_e	Locked	Free point
Driving road (i, j)	The road and its arrival time at j	Node j, at that arrival time
Dwelling at satellite s	The satellite's whole drone plan: victims, trips and order	s, when its last drone lands
Reloading at the depot	The reload	Depot, at the end of the reload
At the depot, idle or not yet activated	Nothing	Depot, now

Only the current road is locked: a truck heading to a satellite along several roads can be re-routed at the next junction. Once a truck reaches a satellite, that satellite's victims and drone plan are frozen until its last drone lands.

3.3.3 Residual problem

The residual problem $P(t_e)$ is model (1)–(38) restricted to what remains:

- **Trucks** start at their free points (h_k, φ_k), with the parcels still on board. An activated truck keeps $\tau_k = 1$ and its activation cost is sunk; an idle truck at the depot may still be activated, with $\alpha^{k1} \geq t_e$.
- **Satellites** are the unvisited candidates, plus each active satellite, pinned first on its truck.
- **Victims** are those not yet served and not at an active satellite. Any of them may move to another satellite that covers it.
- **Objectives** count only what is still open: f_1 over the remaining victims (served victims' times are fixed), f_2 over new activations, remaining driving and remaining drone energy.

The first truck arrival is propagated from the free point instead of the depot:

$$A_j \geq \varphi_k + c_{h_k j}(t_e) - M\,(1 - x^{kr}_{h_k j})$$

The drone level is re-planned with the trucks. A satellite whose victim set changes, or whose route position changes the weight $\Omega_{>p}$ waiting behind it, gets its drone plan re-solved. Plans at satellites with critical victims are re-ordered so critical drops come first when deadlines are tight.

3.3.4 Executing one plan of the front

At each boundary the method returns an archive $\mathcal{A}(t_e)$ approximating the front of $P(t_e)$, but trucks can follow only one plan. The decision maker states a preference $\omega = (\omega_1, \omega_2)$, $\omega_1 + \omega_2 = 1$, once for the whole operation, and the plan executed is

$$x^{\text{exec}}(t_e) = \arg \min_{x \in \mathcal{A}(t_e)} \max_{i=1,2} \omega_i \frac{f_i(x) - z_i^I}{z_i^N - z_i^I}$$

where z^I and z^N are the ideal and nadir points of the $t = 0$ archive, fixed for the whole operation so that ω means the same thing every hour. The default $\omega = (0.5, 0.5)$ weighs normalized waiting and cost equally. Running the operation for several preferences $\omega_1 \in \{0, 0.1, \ldots, 1\}$ gives one realized (f_1, f_2, CV) per preference; the non-dominated ones form the **realized front**, on which strategies are compared (Section 5.2.1).

3.3.5 One re-planning step

1. **Simulate** the executed plan up to t_e, and record every delivery time T_v.
2. **Freeze** served victims, visited satellites, roads driven or occupied, and drone trips in the air; compute each truck's free point.
3. **Update** the states of unfrozen zone roads and recompute fastest paths.
4. **Re-evaluate** the archive under the new times, dropping plans that contradict what was executed.
5. **Choose** a re-planning level: L0 path update, L1 repair, or L2 multi-objective re-optimization (Section 4.3), by a fixed rule or the learned policy of Section 4.4.
6. **Select** the plan to execute by ω and commit it. Trucks at their free points start their next road, which freezes it.
7. Continue to t_{e+1}.

By Proposition 1, every residual problem has a plan that serves everyone, so full coverage is never at risk during execution; what changes from step to step is timeliness and cost.

4. Learning-assisted multi-objective ALNS for the dynamic problem

We solve each residual problem $P(t_e)$ by decomposition, with a method we call L-MOALNS. A fast drone solver gives every satellite a small menu of feasible drone plans. A multi-objective ALNS over the fleet, the truck trips and the victim assignments calls it through a cache and keeps an archive of non-dominated plans. The same search builds the first front at $t = 0$ and is warm-started from the previous archive at every full re-plan. Two double DQNs decide how much to re-optimize each hour and which operators to apply (Fig. 3).

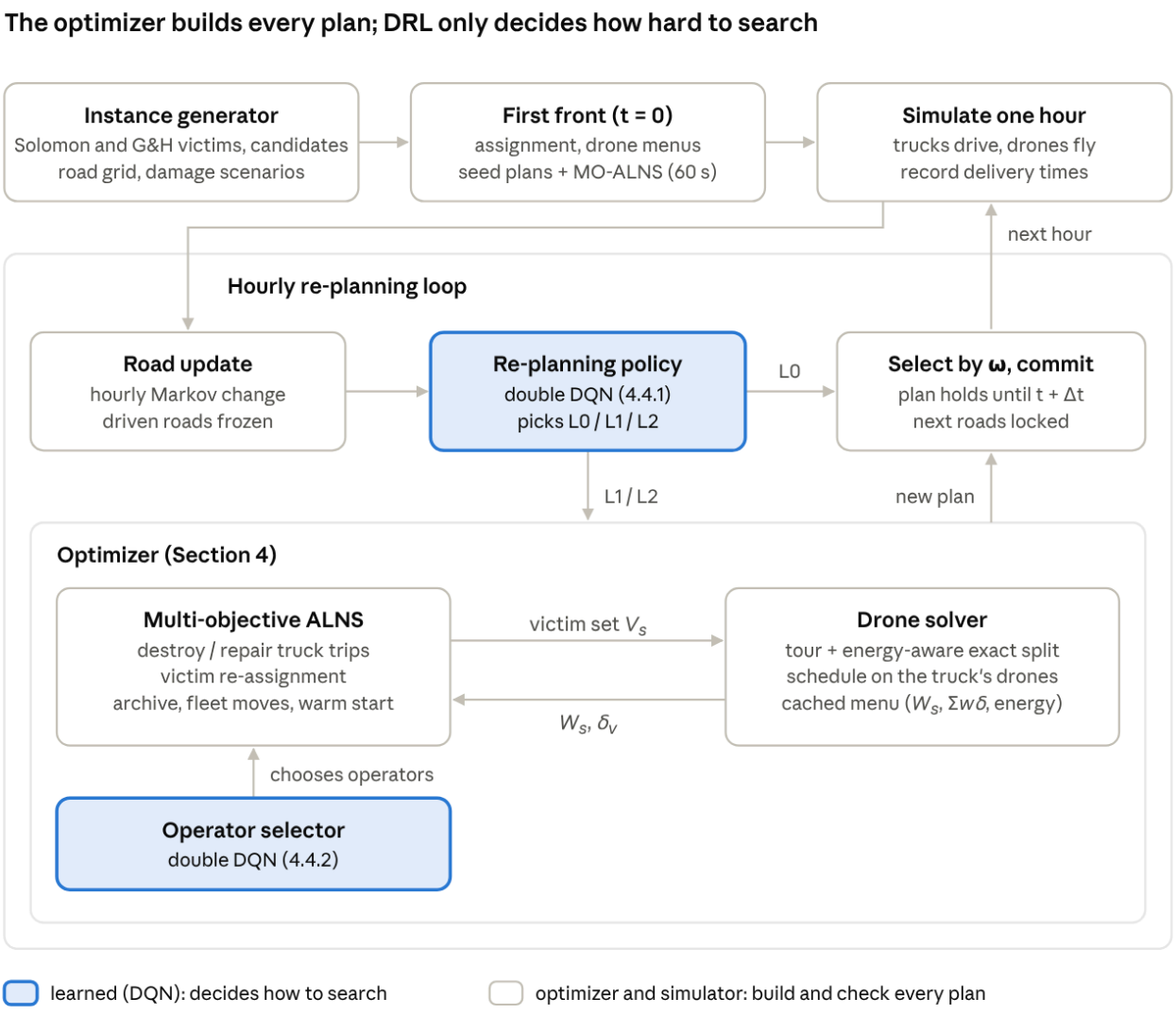


Fig. 3. Solution framework. After the first front, each hour runs the same loop: roads change, the re-planning policy picks a level, the optimizer updates the archive (the ALNS calling the drone solver for every satellite it changes), and one plan is selected by the preference ω and committed until the next hour. The two learned components, shaded, only choose levels and operators.

4.1 Encoding and drone subproblem solver

Encoding. A plan has four parts:

- **Fleet:** the activated trucks and, for each, the number of activated drones $u_k \le \bar{U}$;
- **Truck sequences:** one stop list per truck across all its trips, with the depot as a reload separator, for example (0, s_1, s_2, 0, s_3, 0). An active satellite is pinned first;
- **Assignment:** a covering satellite label for every unserved victim;
- **Drone plans:** for each used satellite, one plan from its menu.

Every encoded plan decodes to a solution of (4)–(38): capacities are checked per trip, coverage is respected by the labels, and every menu plan is battery-feasible. The search therefore never visits an infeasible plan, unlike the constraint-tolerance approach of Yu et al. (2027).

Drone solver. For a satellite s with victims V_s, served by the u_k activated drones of its truck k:

1. **Giant tour.** One tour through V_s (nearest neighbour, then 2-opt). A critical-first variant puts critical victims at the front.
2. **Energy-aware exact split.** Dynamic programming cuts the tour into consecutive trips of at most $Q_2 = 3$ victims (Prins 2004). A cut is allowed only if the trip passes the battery check of Section 3.2.4. Each trip has a duration $d_\rho = \Sigma_i\, 60\ell_i / v_a + m\kappa$ and an energy e_ρ E. The DP is solved twice, minimizing total duration and minimizing total energy, which differ under the rotor model because heavy multi-drop trips cost more energy per kilometre. With at most three victims per trip, each split takes O(3m) time.
3. **Schedule on the u_k drones.** Batteries are swapped instantly, so each drone flies its trips back to back. Two orders are tried: longest trip first, for a short dwell W_s, and critical-first or highest weight per minute (Smith's rule), for early drops to urgent victims.
4. **Menu.** Keep the plans that are non-dominated in (W_s, $\Sigma\, w_v\delta_v$, energy), usually two to five.

The truck search picks, at each satellite, the menu plan that minimizes the current scalarization at the satellite's route position: $\Omega >_p W_s + \Sigma\, w_v\delta_v$ for f_1 and $c_a \times$ energy for f_2. Menus are cached by (satellite, victim set, drones); a victim move changes only two entries, so most calls are cache hits.

4.2 Multi-objective ALNS

Archive. The search keeps an archive $\mathcal{A}$ of at most 30 plans that are non-dominated under the constrained dominance of Section 3.2.1. When it overflows, the plan with the smallest crowding distance is removed (Deb et al. 2002). Every plan the search evaluates is offered to the archive.

Decomposition into scalar searches. Following MOEA/D (Zhang & Li 2007), the search cycles through 11 weight vectors $\omega^h = (h/10,\ 1 - h/10)$. Each segment of 100 iterations works for one ω^h: it starts from the archive plan that is best for ω^h and minimizes

$$g(x \mid \omega^h) = \Pi\, CV(x) + \max_{i=1,2}\ \omega_i^h\, \frac{f_i(x) - z_i^I}{z_i^N - z_i^I}$$

with $\Pi = 10^4$, so CV dominates as in constrained dominance. The ideal and nadir points are updated from the archive as the search runs. Acceptance is simulated annealing on g.

Destroy operators (remove 10–30% of the used satellites):

- random removal;
- worst removal: the largest weighted-delay contribution $\Omega_s \cdot A_s$, and satellites with critical victims close to their deadline;
- related removal, by distance;
- **damage-related removal:** satellites whose paths use roads that changed state at this step;
- trip removal: a whole truck trip;
- **truck closure:** all satellites of one truck, which is deactivated.

Repair operators. Greedy and regret-2 insertion by the change in g. A satellite may join an existing trip, open a new trip after a reload, or open an idle truck. Satellites are inserted in decreasing Ω_s, critical ones first.

Victim moves. Re-assign a victim to another covering satellite, or swap two victims between satellites. Each move re-solves two drone menus through the cache and checks trip capacity.

Fleet moves. Add or remove one drone on a truck, which re-solves the menus of that truck's satellites. Together with truck closure and opening, these moves let the search travel along the cost axis of the front.

Local search after each repair: relocate, swap, 2-opt*, Or-opt, and moving a satellite between trips.

Fast evaluation. For each truck sequence we store prefix arrivals, dwell ends and suffix weights $\Omega_{\geq p}$. Inserting satellite s between stops a and b at position p shifts every later arrival by $\delta = c(a, s) + W_s + c(s, b) - c(a, b)$, so

$$\Delta \sum_v w_v T_v = \Omega_s \big(A_a + W_a + c_{as}\big) + \delta\, \Omega_{\geq p} + \sum_{v \in V_s} w_v \delta_v$$

$$\Delta f_2 = c_t \big(c_{as} + c_{sb} - c_{ab}\big) + c_a\, \Delta E_s$$

both in O(1). Depot reloads are stops of weight 0, so the same formulas cover multi-trip routes. For deadlines, each position stores its forward slack (Savelsbergh 1992): a shift δ within the slack leaves CV unchanged, again in O(1). Each trip stores its parcel count, so capacity is checked in O(1).

Initial archive. Victims go to their nearest covering satellite, then a greedy pass moves victims from crowded satellites to their second coverer. Regret-2 insertion then builds one plan for each fleet size from one truck to |K| trucks, with all drones and with one drone per truck, which seeds the archive across the cost axis.

Budgets. 60 s for the first front at t = 0, and 2 s or 10 s per full re-plan (Section 4.4).

4.3 Re-planning levels

After each road update, the executed plan and the archive are re-evaluated under the new times. Let Δg be the relative increase of the executed plan's g(x | ω) at the decision maker's preference ω.

Level	Rule	Action
L0 — path update	Δg ≤ 1% and no new critical deadline at risk	Keep every sequence; only road paths and times change
L1 — repair	Δg ≤ 5%	0.5 s of local and victim moves around the changed roads, offered to the archive
L2 — re-optimize	Δg > 5% or a critical deadline at risk	MO-ALNS warm-started from the re-evaluated archive, 2 s or 10 s

Archive plans that contradict what was executed (a satellite already visited by another truck, a victim already served) are repaired by pinning the executed parts, or dropped if repair fails.

4.4 Learning-assisted control

Deep reinforcement learning decides **how** to re-optimize, not the routes. Learning routes end to end would need masks for every hard constraint (capacity, battery, coverage, frozen roads, locked drone trips) and would still guarantee nothing; disaster data are scarce; and a policy that only picks levels and operators can cost quality or time, never feasibility.

4.4.1 Re-planning policy

At each boundary t_e the policy picks the level that the rule of Section 4.3 picks.

State (normalized features):

- the preference ω being executed;
- the time t_e relative to the expected end of operations;
- the share of victims still unserved, per priority class;
- the number of critical victims at risk and the smallest critical slack;
- unvisited satellites, parcels still on trucks, trucks in each mode, and idle trucks;
- changed roads on the planned paths, and the share of planned driving on damaged or closed roads;
- Δg after an L0 update and after an L1 repair, both cheap to compute;
- the archive's size and hypervolume change after the road update;
- the share of closed roads in the zone, and the computing time used so far.

Actions: L0, L1, L2 with a 2 s budget, L2 with a 10 s budget.

Reward. The realized change of the scalarized objective during $(t_e, t_{e+1}]$, minus a computing penalty:

$$r_e = -\left[\Pi\,\Delta CV + \omega_1 \frac{\Delta f_1}{z_1^N - z_1^I} + \omega_2 \frac{\Delta f_2}{z_2^N - z_2^I}\right]_{(t_e,\,t_{e+1}]} - \lambda_{\text{cpu}}\,\text{CPU}_e$$

Realized increments add up over an episode, so the return is the negative of the realized scalarized outcome plus total computing cost. The reward uses the weighted sum because the Tchebycheff function is not additive over time.

One policy for all preferences. Each training episode samples ω_1 uniformly in [0, 1] and feeds ω to the network, so a single preference-conditioned Q-network serves every point of the realized front (Abels et al. 2019).

Learning. A double DQN (Mnih et al. 2015; van Hasselt et al. 2016) with experience replay, a target network and ε-greedy exploration, on a network of two hidden layers of 64. One episode is one simulated operation on a training instance and road seed. The fixed rule of Section 4.3 is both the baseline and the warm-up policy that fills the replay buffer.

4.4.2 Operator selection inside the ALNS

Inside an L2 re-plan, each iteration chooses a (destroy, repair) pair: 6 × 2 = 12 actions.

- **State:** the fraction of the budget used, the annealing temperature, iterations since the last improvement, the last pair, the gap between the current and segment-best g, the current weight vector ω^h, and whether the re-plan was triggered by damage, by a deadline at risk, or both.

- **Reward:** the relative improvement of $g(x \mid \omega^h)$, plus 1 when the candidate enters the archive.

A DQN trained across instances replaces the adaptive weights of Ropke & Pisinger (2006). It can specialize, for example preferring damage-related removal right after large road changes and truck closure in cost-leaning segments.

4.4.3 Training and evaluation

- **Split:** policies train on road seeds kept apart from the test set (Section 5.1.1).
- **Generalization:** train at medium intensity, test at low, medium and high, and on unseen layouts.
- **Comparisons:** the fixed rule, always running L2, and the full-information oracle (Section 5.4.3).

4.5 Algorithm framework

Algorithm 1. L-MOALNS for the dynamic problem.

Input: an instance, the road process, a preference ω, the two policies (or the rule and adaptive weights).

1. At $t = 0$, build the initial archive (Section 4.2) and run MO-ALNS for 60 s to get $\mathcal{A}(0)$. Fix the normalization points z^I and z^N from $\mathcal{A}(0)$.
2. Select $x^{exec}(0)$ by ω (Section 3.3.4) and commit it.
3. For $e = 1, 2, \ldots$ until every victim is served and every truck is back:
    1. Simulate the executed plan to t_e; freeze served victims, visited satellites, driven and occupied roads, and trips in the air.
    2. Update road states and fastest paths.
    3. Re-evaluate and repair the archive; compute Δg and the policy features.
    4. Choose a level with the re-planning policy (or the rule).
    5. Apply L0, L1 or L2, the L2 search choosing operators with the operator policy (or adaptive weights), to get $\mathcal{A}(t_e)$.
    6. Select $x^{exec}(t_e)$ by ω and commit it.
4. *Output:* the realized delivery times, f_1, f_2 and CV, and the archive at every step.

For the realized front, steps 1–4 run once per preference $\omega_1 \in \{0, 0.1, \ldots, 1\}$ on the same road history.

5. Computational experiments

This section sets out the planned study; no results are reported yet. It follows the experimental structure of Yu et al. (2027) and tests five claims:

- the drone echelon serves victims sooner than a ground echelon at comparable cost, more so as damage grows (5.3);
- L-MOALNS approximates exact fronts on small instances and beats evolutionary baselines on large ones (5.4.1, 5.4.2);
- hourly re-planning of both echelons gives better realized fronts than a static plan, and the learned policy matches always re-optimizing at a fraction of the time (5.4.3);

- each component matters (5.5);
- conclusions are robust to drone technology and disruption settings (5.6, 5.7).

5.1 Instance description and baseline algorithms

5.1.1 Benchmark instances

Instances are built from two classic VRPTW benchmarks: Solomon (1987), 25 to 100 customers, and Gehring & Homberger (1999), 200 to 1,000 customers. Every instance is guaranteed feasible, with every victim within single-trip drone range of at least two candidate satellites, and a validation report checks this (Fig. 4).

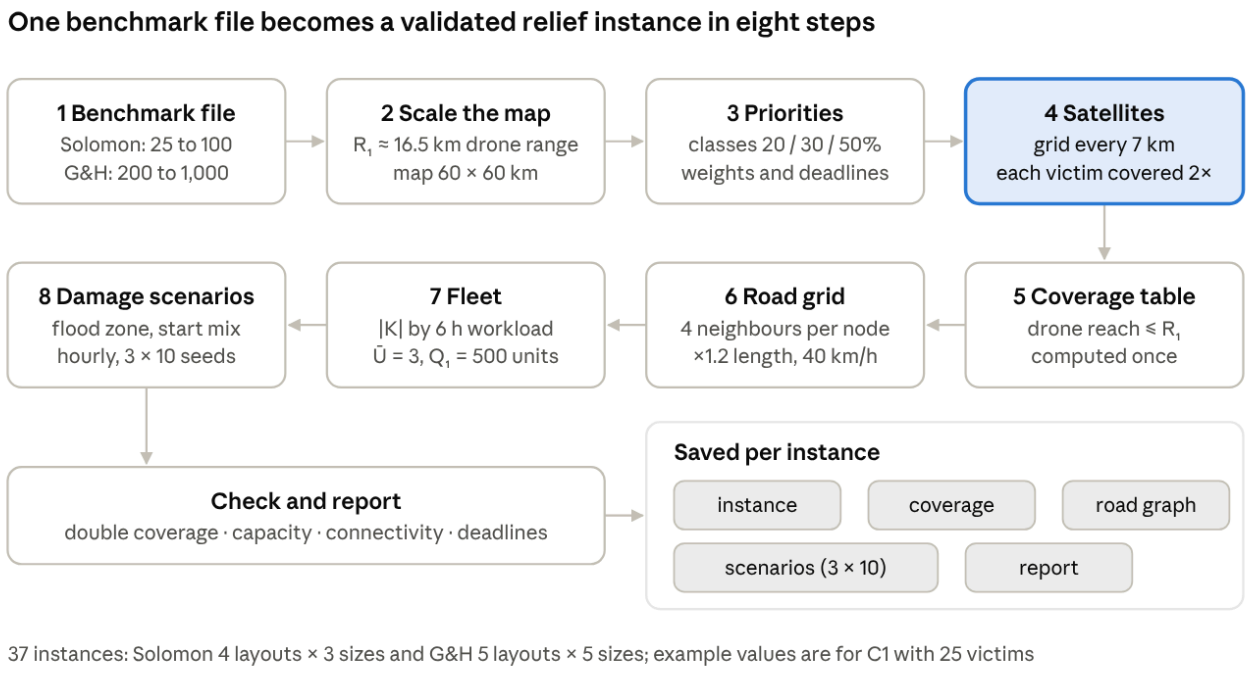


Fig. 4. Data generation pipeline. The steps run left to right on the top row and right to left on the bottom row. Step 4, the double-coverage placement of satellites, is what guarantees every instance is feasible.

Base data.

- **Solomon:** n = 25, 50 or 100 victims (the first n customers, as in Solomon's subsets) on the four distinct layouts C1, C2, R1 and RC1. R2 and RC2 share the coordinates of R1 and RC1.
- **Gehring & Homberger:** n = 200, 400, 600, 800 or 1,000 victims on the five distinct layouts C1, C2, R1, R2 and RC. RC1 and RC2 share coordinates, and the ten instances of a class differ only in time windows, so we use instance 1 of each class. Files come from the PyVRP instance repository.
- **Kept:** the depot and customer coordinates. **Dropped:** demands (each victim needs one unit), time windows and service times.

Scale. Every map is scaled to the same 60 km × 60 km region: 600 m per unit for Solomon, and 430 m down to 120 m for Gehring & Homberger as the side grows from 140 to 500. With $R_1 \approx 16.5$ km, the drone range is about 0.27 of the side (a 43 km variant gives 0.38). Larger instances therefore mean more people in the same flooded region, not a larger

region; keeping 600 m per unit would stretch the 1,000-victim maps to 300 km and call for about 57 trucks.

Priorities. Classes are drawn at random: 20% critical, 30% high, 50% normal. In a variant, victims closer to the flood centre are more likely to be critical.

Candidate satellites (double coverage).

1. **Grid.** Candidates lie on a grid of spacing $\rho R_1 / \sqrt{2} \approx 7.0$ km, with $\rho = 0.6$, so every point has the four corners of its cell within $\rho R_1 \approx 10$ km.
2. **Selection.** Choose the fewest candidates such that every victim is within ρR_1 of at least two. This set multicover problem is solved with CP-SAT in seconds. Candidates inside an optional no-go core of the flood zone are excluded first.
3. **Margin.** A full three-unit trip reaches a cluster about 13.0 km away under the rotor model, so the 10 km planning radius leaves battery for the detours between drops.

Fleet.

- **Capacity.** With $Q_1 = 500$ units, capacity never binds: the fleet carries 3 to 40 times total demand, as is realistic for light relief items. A **stress-test variant** sets $Q_1 = \lceil n / (1.5|K|) \rceil$, so trucks must make second trips.
- **Drones.** Each truck can carry $\bar{U} = 3$ drones; how many are activated is a decision on the front.
- **Trucks available.** $|K|$ is sized by workload. The generator estimates the undamaged work (total drone dwell with each victim at its nearest covering satellite, three units per trip and three drones, plus one nearest-neighbour truck tour over the used satellites at ×1.2 circuity and 40 km/h) and sets $|K| = \max(2, \lceil \text{work} / 6\ \text{h} \rceil)$. How many of them are activated is again a decision on the front.

Table 4. Candidate satellites and available trucks, from a prototype of the placement and sizing steps on the benchmark files with the rotor power model.

Set	Layout	Victims	Candidate satellites	Trucks available
Solomon	C1	25 / 50 / 100	5 / 9 / 19	2 / 2 / 2
Solomon	C2	25 / 50 / 100	7 / 12 / 19	2 / 2 / 3
Solomon	R1	25 / 50 / 100	12 / 18 / 20	2 / 2 / 3
Solomon	RC1	25 / 50 / 100	6 / 11 / 23	2 / 2 / 3
G&H	C1	200 / 400 / 600 / 800 / 1,000	25 / 30 / 29 / 33 / 35	3 / 4 / 5 / 5 / 6
G&H	C2	200 / 400 / 600 / 800 / 1,000	29 / 27 / 29 / 29 / 29	4 / 4 / 5 / 5 / 6
G&H	R1	200 / 400 / 600 / 800 / 1,000	38 / 39 / 40 / 42 / 44	4 / 5 / 6 / 6 / 7
G&H	R2	200 / 400 / 600 / 800 / 1,000	38 / 39 / 40 / 42 / 44	4 / 5 / 6 / 6 / 7
G&H	RC	200 / 400 / 600 / 800 / 1,000	35 / 36 / 36 / 40 / 44	4 / 5 / 6 / 6 / 7

With the region fixed, candidates level off at 25 to 44 and victims per satellite grow instead: a median of about 4 for Solomon and 22 to 33 at 1,000 victims (up to 86). The optimizer decides which candidates are used and how many victims each serves.

Road graph. Roads follow the candidate grid (about 75 junctions): each grid node links to its four neighbours, and the depot to its four nearest nodes. Base times are the Euclidean

length × 1.2 / v_t. A grid has no bridge edges, so every satellite has at least two separate paths to the depot.

Flood zone and scenarios. The zone is centred on a random victim (Section 3.3.1). For each instance, intensity and seed, the full hourly road history is generated in advance and stored, so every method faces the same realization. There are 10 seeds per instance and intensity, 3 of them kept for training the policies.

Deadline check. For each critical victim, a lower bound on its delivery time is the fastest drive at t = 0 to its nearest covering satellite plus one direct drone trip. If this exceeds D_v, the deadline becomes ⌈1.2 × bound⌉ and the change is logged, so each critical deadline is reachable on its own.

Size. 12 Solomon instances (4 layouts × 3 sizes) and 25 Gehring & Homberger instances (5 layouts × 5 sizes), 37 in total, each with 3 intensities × 10 seeds: 1,110 scenarios. For the exact comparison of Section 5.4.1, small instances with n = 10 and 15 are cut from the Solomon C1, R1 and RC1 files in the same way.

5.1.2 Comparison algorithms

Static fronts (the problem at t = 0, equal computing time per method):

Method	Role	Notes
ε-constraint + Gurobi	Exact reference	Linearized MILP of Section 3.2.4; f_2 bounded at 11 levels; 1 h per subproblem; n = 10 and 15 (Haimes et al. 1971)
NSGA-II	Evolutionary baseline	Our encoding, with the ALNS moves as mutation and constrained dominance (Deb et al. 2002)
MOEA/D	Decomposition baseline	Tchebycheff, 11 weight vectors, same moves (Zhang & Li 2007)
DSDE	State of the art for 2E-VRPLDTW	Public code of Yu et al. (2027), adapted to our constraints: waiting truck, multi-trip trucks, coverage and unit loads
Weighted-sum ALNS	Single-objective baseline	One ALNS run per weight of f_1 and normalized f_2
MO-ALNS (ours)	Proposed	Adaptive operator weights or learned selector

Dynamic strategies (realized fronts on the same road histories):

Strategy	Description
Static	Front at t = 0; trucks follow the executed plan on fastest paths at current road states, but never re-sequence
Always L2	Full re-optimization (10 s) at every step
Rule	Levels by the rule of Section 4.3
DRL policy	Levels by the double DQN of Section 4.4.1
DRL policy + DRL operators	As above, with the learned operator selector inside L2
Oracle	Knows the whole road history at t = 0 and plans once with a 10× budget (a bound)

Delivery systems (Section 5.3), with the same first echelon:

- **Drones** (ours);
- **Ground couriers:** motorbikes at each satellite, 10 units per trip, 30 km/h, on road distances ×1.3 and slowed by the flood-zone state of the nearest road;
- **Trucks only:** trucks stop at the road junction nearest each victim cluster, with no second echelon.

5.1.3 Time complexity analysis

Let n be the number of victims, $\bar{m}$ the victims per used satellite, $|S|$ the candidates, $A \le 30$ the archive size, and $G_r = (N_r, E_r)$ the road graph.

- **Drone solver**, per satellite: $O(\bar{m}^2)$ for one 2-opt pass, $O(Q_2\, \bar{m})$ for the split, $O(\bar{m} \log \bar{m})$ for scheduling. Results are cached.
- **One ALNS iteration:** $O(|S|)$ to destroy, $O(|S|^2)$ to reinsert with O(1) move evaluation, $O(\bar{m}^2)$ per uncached menu, and O(A) to update the archive.
- **Road update**, per step: one Dijkstra from the depot and from each satellite, $O(|S|\ |E_r| \log |N_r|)$; with about 75 junctions and 150 roads this takes milliseconds.
- **Policies:** a forward pass through a 2 × 64 network, microseconds.
- **Exact MILP:** $O(|U|\ \bar{H}\ (|S| + n)^2\ Q_2)$ binaries, which limits it to $n \le 15$.

Evolutionary methods such as NSGA-II and DSDE decode every individual of the population in full at each generation, at least $O(n + |S|^2)$ per individual; the ALNS changes a few satellites at a time and re-evaluates them in O(1), which is what makes hourly re-planning within seconds possible.

5.2 Performance metrics and experimental setup

5.2.1 Performance metrics

Front quality. Objectives are normalized by the ideal and nadir points of the union of all fronts on an instance.

- **Hypervolume (HV)** of the front with reference point (1.1, 1.1), the main metric, as in Yu et al. (2027) (Zitzler & Thiele 1999);
- **IGD+** to the reference front: the exact front where Gurobi finishes, otherwise the non-dominated union of all methods (Ishibuchi et al. 2015);
- the number of non-dominated plans, and CV* of the front.

For the dynamic strategies, the same metrics are computed on the **realized front** of Section 3.3.4.

Relief metrics, reported at the default preference $\omega = (0.5, 0.5)$:

- critical victims served late, and total critical lateness (CV);
- priority-weighted average waiting time (f_1), and the average and maximum T_v per class;
- makespan: when the last victim is served;
- fleet activated, truck driving time, truck trips, total dwell and drone energy (f_2 and its parts);
- the lowest battery level reached per trip;
- re-planning: the share of steps at each level, and CPU time per step;

- plan stability: satellites moved between trucks or trips per re-plan.

Statistics. Comparisons are paired, with the same instance and road seed for every method. Pairs are tested with Wilcoxon signed-rank tests at $\alpha = 0.05$, and several methods with the Friedman test and Nemenyi post-hoc ranks.

5.2.2 Experimental platform and parameter settings

The planned implementation is in Python, with Numba for move evaluation, PyTorch for the policies, OR-Tools CP-SAT for the generator and Gurobi for the exact fronts, one thread per run. Model parameters are those of Table 2. Method settings follow; values marked "proposed" are starting points, to be tuned on the training seeds.

Table 5. Default settings of the method.

Group	Setting	Default	Note
Objectives	Priority weights w_v (critical / high / normal)	5 / 2 / 1	Section 3.1
Objectives	Targets D_v	120 / 240 / 480 min	Critical: constraint violation
Objectives	Lateness weight π_l	2	Proposed
Objectives	CV weight Π in scalarizations	10^4	Larger than any normalized objective
Objectives	Executed preference ω	(0.5, 0.5); 11 values for realized fronts	Section 3.3.4
Re-planning	Time step Δt	1 h (30 min variant)	Section 3.3
Re-planning	Level thresholds	1%, 5% of g	Section 4.3
Re-planning	Budgets	60 s at t = 0; L1 0.5 s; L2 2 s or 10 s	Sections 4.2–4.3
MO-ALNS	Archive size	30, crowding-distance truncation	Deb et al. (2002)
MO-ALNS	Weight vectors, segment length	11, 100 iterations	Zhang & Li (2007)
MO-ALNS	Destroy size	10–30% of used satellites	Section 4.2
MO-ALNS	Operator scores (new best / improving / accepted)	33 / 9 / 13; update every 100 iterations, reaction 0.1	Ropke & Pisinger (2006)
MO-ALNS	Annealing	Start: a 5% worse g accepted with probability 0.5; geometric cooling to 0.1%	Proposed
Drone solver	Units per trip; battery check	≤ 3; $e_\rho \leq 80\%$	Section 3.2.4
Drone solver	Menu size	2–5 non-dominated plans	Section 4.1
Re-planning policy	Network	2 × 64, ReLU; input includes ω	Section 4.4.1
Re-planning policy	Learning rate, batch	10^{-3} (Adam), 64	Proposed
Re-planning policy	Discount γ	1.0	Finite episodes; return = realized

Group	Setting	Default	Note
			outcome
Re-planning policy	Replay buffer, target update	50,000 transitions, every 500 updates	Proposed
Re-planning policy	Exploration ε	1.0 → 0.05 over 2,000 episodes	Proposed
Re-planning policy	Training	5,000 episodes, ω_1 \~ U[0, 1]; first 200 by the rule	Proposed
Re-planning policy	Computing penalty λ_cpu	0.01 per CPU second	Proposed
Operator selector	Network, actions	2 × 64, 12 (destroy, repair) pairs	Section 4.4.2
Operator selector	Learning rate, batch, γ	10^{-3}, 64, 0.9	Proposed
Operator selector	Replay, exploration	20,000 transitions, ε 0.3 → 0.05	Proposed
Generator	Coverage	≥ 2 candidates within ρR_1, ρ = 0.6 (≈ 10 km)	Section 5.1.1
Generator	Fleet sizing	Undamaged work ≤ 6 h per truck	Section 5.1.1
Generator	Flood zone, priority mix	About 30% of roads; 20 / 30 / 50%	Sections 3.3.1, 5.1.1
Generator	Seeds	10 per instance and intensity, 3 for training	Section 5.1.1

5.3 Comparison among relief delivery systems

With the same first echelon, the three systems of Section 5.1.2 (drones, ground couriers, trucks only) are compared on all 37 instances at the three intensities. We report realized fronts and the relief metrics at ω = (0.5, 0.5). The expected result: drones serve victims sooner at a modest energy cost, and their advantage grows with damage, because couriers and trucks share the failing roads while drones do not.

5.4 Comparison of solution methods

5.4.1 Exact fronts on small instances

On instances with n = 10 and 15 (Solomon C1, R1, RC1; t = 0, medium damage), the ε-constraint method with Gurobi gives the exact front of the linearized MILP. L-MOALNS and the baselines run for 60 s, 10 times each. We report IGD+ to the exact front, the share of exact Pareto points recovered, and Gurobi's time and gap where it stops at the limit.

5.4.2 Heuristics and metaheuristics on the full benchmark

All static methods of Section 5.1.2 run on the 37 instances at t = 0, 10 runs each, with equal time. We report HV and IGD+ per instance group (Solomon, and Gehring & Homberger by size), Friedman ranks, and how the gap changes from 25 to 1,000 victims.

5.4.3 Dynamic strategies

The six strategies of Section 5.1.2 run on the 1,110 scenarios (test seeds only for the learned ones). We report realized-front HV, the relief metrics at the default preference, CPU

per step, and plan stability. The oracle's gap shows how much is lost to not knowing future road states.

5.5 Ablation experiments

Run with the Rule strategy unless stated.

Component	**Variants**	**Question**
Energy model	Rotor power vs. linear (plan with each, evaluate with the rotor model)	How many linear-model trips land below the 20% reserve, and how the front shifts
Objective form	Pareto front vs. one lexicographic weighted sum	What the front shows that a single weighted plan hides
Fleet decisions	Activation on the front vs. full fleet always	How much cost is saved at equal waiting time
Fleet moves	On / off	Whether truck closure and drone moves are needed to cover the cost axis
Drone-level re-planning	On / off (assignment and drone plans frozen at t = 0)	How much of the gain comes from re-planning the drones
Critical deadlines	Constraint violation vs. soft lateness	How many critical misses CV prevents, and at what cost to other victims
Multi-trip trucks	$Q_1 = 500$ vs. stress test, with and without second trips	What second trips save when capacity is tight
Drones per truck $\bar{U}$	1, 2, 3, 4	How much dwell and waiting fall per extra drone
Executed-plan rule	Tchebycheff with ω vs. knee point	Whether a parameter-free rule is enough
Re-planning control	Rule, random, always L2, DQN, double DQN, dueling double DQN, PPO	Whether learning helps, and which algorithm
Operator selection	Uniform, adaptive weights, double DQN	Whether a learned selector beats adaptive weights
Preference conditioning	One conditioned policy vs. one policy per ω	Whether one network can serve the whole front

5.6 Sensitivity of drone technological parameters

Following Yu et al. (2027), we vary one drone parameter at a time, keep the candidate satellites, and recompute coverage:

Parameter	**Values**	**Effect to measure**
Battery E	150, 180, 220 Wh	Coverage radius, trip sizes, dwell
Payload Q_2	2, 3, 4 units	Whether larger trips pay off under convex power
Drone mass W	5, 6, 7 kg	Energy per trip and range
Rotor disc area ζ	0.20, 0.26, 0.32 m^2	Power level
Drone speed v_a	50, 60, 70 km/h	Dwell vs. range; the hover-power model ignores drag, so this is reported with that caveat

Parameter	Values	Effect to measure
Drop hover κ	0.1, 0.5, 1 min	Energy and dwell of multi-drop trips

When a setting leaves some victim with fewer than two covering candidates, that is reported rather than repaired.

5.7 Sensitivity under disruption settings

Factor	Values	Question
Damage intensity	Low, medium, high	How fronts and the value of re-planning change with damage
Closed-road multiplier	×5, ×10	How sensitive results are to closure severity
Time step Δt	1 h, 30 min	Whether re-planning more often pays off
Flood-zone size	20%, 30%, 40% of roads	How far damage must spread before re-planning matters
Range share	0.27, 0.38 of the map side	How drone range relative to the map changes satellite use

6. Conclusion and prospects

This paper formulates a dynamic two-echelon truck–drone relief routing problem as a bi-objective MO-MINLP. Trucks make several trips to staging satellites and wait while their drones fly multi-drop trips under rotor power; the model trades priority-weighted waiting time against the trucks, drones and energy committed, with critical-deadline misses minimized first. Unit demands give the model an exact MILP form for small instances. Hourly Markov road damage is handled by rolling re-planning that freezes what is under way, executes one plan of the current front by preference, and keeps full coverage feasible at every step by construction. The solution method, L-MOALNS, combines an energy-aware drone split, an archive-based ALNS with fleet moves, and preference-conditioned double DQNs that control the re-planning effort and the operators.

Compared with the static 2E-VRPLDTW of Yu et al. (2027), the model adds road dynamics, a relief objective, the truck's wait for its drones, multi-trip trucks and guaranteed coverage; it takes from them the rotor power model and the Pareto view. The experimental design of Section 5 will test whether these choices pay off.

Limitations and prospects, in the order we would address them:

- **Cross-satellite recovery.** Drones land at their own truck. Letting a drone land at another satellite where a truck waits, as Yu et al. (2027) allow, could shorten dwells when satellites are close.
- **Demand changes.** Demand is fixed at one parcel per victim. Growing demand or newly reported victims could re-open the not-yet-launched trips of an active satellite.
- **Partial information.** Road states are known when they change. Noisy reports and a belief over road states would call for robust or stochastic re-planning.
- **Drone physics.** The hover-power model ignores speed, drag, wind and no-fly zones.

- **Battery logistics.** Swaps are instant and spares unlimited. A finite battery stock charged on the truck would add a resource constraint.
- **Satellites.** Candidate sites never fail. A satellite that floods could be handled by the same re-assignment moves.
- **More objectives.** Equity between areas could be a third objective, with a three-dimensional front.

References

- Abels, A., Roijers, D. M., Lenaerts, T., Nowé, A., & Steckelmacher, D. (2019). Dynamic weights in multi-objective deep reinforcement learning. *Proceedings of the 36th International Conference on Machine Learning*, PMLR 97, 11–20.
- Chen, X., Ulmer, M. W., & Thomas, B. W. (2022). Deep Q-learning for same-day delivery with vehicles and drones. *European Journal of Operational Research*, 298(3), 939–952. Preprint: arXiv:1910.11901
- Deb, K., Pratap, A., Agarwal, S., & Meyarivan, T. (2002). A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation*, 6(2), 182–197. doi
- Dorling, K., Heinrichs, J., Messier, G. G., & Magierowski, S. (2017). Vehicle routing problems for drone delivery. *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, 47(1), 70–85. doi
- Du, W., Wang, J., Liu, J., Wang, H., Zhou, J., Mei, C., & Pang, Y. (2026). Optimization of emergency supplies allocation for urban flood disasters. *Results in Engineering*, 32, 112619. doi
- Duan, H., Li, X., Feng, Y., Zhang, G., Lu, Q., & Meng, X. (2026). Multi-objective hyper heuristic memetic algorithm for time-dependent vehicle routing problem with drone considering simultaneous pickup and delivery. *Expert Systems with Applications*, 297, 129196. doi
- Gehring, H., & Homberger, J. (1999). A parallel hybrid evolutionary metaheuristic for the vehicle routing problem with time windows. In *Proceedings of EUROGEN99*, 57–64. Instances: SINTEF TOP, files via the PyVRP instance repository.
- Haimes, Y. Y., Lasdon, L. S., & Wismer, D. A. (1971). On a bicriterion formulation of the problems of integrated system identification and system optimization. *IEEE Transactions on Systems, Man, and Cybernetics*, SMC-1(3), 296–297. doi
- Ishibuchi, H., Masuda, H., Tanigaki, Y., & Nojima, Y. (2015). Modified distance calculation in generational distance and inverted generational distance. In *Evolutionary Multi-Criterion Optimization (EMO 2015)*, LNCS 9019, 110–125. doi
- Li, Q., & Gonsalves, T. (2026). A reinforcement learning framework for traveling salesman and vehicle routing problem with drones. *Drones*, 10, 616. doi
- Luo, Y., Chen, Y., & Wahab, M. I. M. (2027). Multi-trip truck-drone routing problem with simultaneous pickup and delivery and temporal constraints. *Transportation Research Part E*, 217, 105235. doi
- Meng, S., & Ma, H. (2026). Electric vehicle routing problem with multi-visit drones considering time windows and battery swapping stations. *Transportation Research Part E*, 215, 105107. doi

- Miettinen, K. (1999). *Nonlinear Multiobjective Optimization*. Kluwer Academic Publishers, Boston.
- Mnih, V., et al. (2015). Human-level control through deep reinforcement learning. *Nature*, 518, 529–533. doi
- Ngueveu, S. U., Prins, C., & Wolfler Calvo, R. (2010). An effective memetic algorithm for the cumulative capacitated vehicle routing problem. *Computers & Operations Research*, 37(11), 1877–1885. doi
- Nguyen, D. V. A., Piplani, R., & Gunawan, A. A two-echelon covering tour vehicle routing problem with drones for post-disaster relief. *arXiv:2609.38227* (under review).
- Peng, W., Wang, D., Yin, Y., & Cheng, T. C. E. (2025). Multi-agent deep reinforcement learning-based truck-drone collaborative routing with dynamic emergency response. *Transportation Research Part E*, 195, 103974. doi
- Prins, C. (2004). A simple and effective evolutionary algorithm for the vehicle routing problem. *Computers & Operations Research*, 31(12), 1985–2002. doi
- Ropke, S., & Pisinger, D. (2006). An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows. *Transportation Science*, 40(4), 455–472. doi
- Savelsbergh, M. W. P. (1992). The vehicle routing problem with time windows: Minimizing route duration. *ORSA Journal on Computing*, 4(2), 146–154. doi
- Shi, Y., Guo, Q., Pan, P., Chen, M., Wu, Y., & Guo, H. (2026). Robust optimization for two-echelon truck and drone collaborative delivery in emergency relief. *Applied Soft Computing*, 201, 115738. doi
- Smith, W. E. (1956). Various optimizers for single-stage production. *Naval Research Logistics Quarterly*, 3(1–2), 59–66. doi
- Solomon, M. M. (1987). Algorithms for the vehicle routing and scheduling problems with time window constraints. *Operations Research*, 35(2), 254–265. doi
- van Hasselt, H., Guez, A., & Silver, D. (2016). Deep reinforcement learning with double Q-learning. *Proceedings of the AAAI Conference on Artificial Intelligence*, 30(1). doi
- Yu, F., Li, C., Chen, Q., Li, Y., Chen, Y., & Yang, X. (2027). Multi-objective two-echelon vehicle-drone routing with load-dependent drone under pickup delivery time windows. *Transportation Research Part E*, 217, 105195. doi. Code: DSDE-2EVRPLDTW
- Zhang, Q., & Li, H. (2007). MOEA/D: A multiobjective evolutionary algorithm based on decomposition. *IEEE Transactions on Evolutionary Computation*, 11(6), 712–731. doi
- Zitzler, E., & Thiele, L. (1999). Multiobjective evolutionary algorithms: A comparative case study and the strength Pareto approach. *IEEE Transactions on Evolutionary Computation*, 3(4), 257–271. doi